

Сравнение производительности SQLite и PostgreSQL

В разрезе работ по Desktop-приложениям возник вопрос выбора БД. Рассматривались два основных варианта: Postgres Portable и SQLite.

Для проведения тестирования на Wasaby Framework версии 3.7.4.100 была написана утилита, которая при помощи библиотеки sbis-dbi выполняла операции сравнения.

Методика тестирования

- Вставка большого числа записей. Выполнялись вставки, сгруппированные в транзакции: по 1, по 10 и по 1000 записей на транзакцию:

```
1 insert into test_table (int_field, another_field) values ($1, $2)
```

- Обновление записей. Выполнялись операции, сгруппированные в транзакции: по 1, по 10 и по 1000 записей в транзакции:

```
1 update test_table set another_field = $1 where int_field = $2
```

- Удаление записей. Выполнялись операции, сгруппированные в транзакции: по 1, по 10 и по 1000 записей в транзакции:

```
1 delete from test_table where int_field = $1
```

- Выборка данных из двух связанных таблиц (left join):

```
1 SELECT * FROM test_tbl1 LEFT JOIN test_tbl2 ON test_tbl1.ref =  
test_tbl2.id2 WHERE test_tbl1.id1 = $1
```

- Выборка данных из таблицы без индекса (seqscan), наполненной большим количеством данных (1000000 записей):

```
1 SELECT * FROM test_idx_tbl where int = $1
```

- Выборка данных из таблицы по индексу (index scan), наполненной большим количеством данных (1000000 записей):

```
1 SELECT * FROM test_idx_tbl where int = $1
```

- Выборка данных из таблицы без индекса (seqscan), наполненной малым количеством данных (1000 записей):

```
1 SELECT * FROM test_idx_tbl where int = $1
```

- Выборка данных из таблицы по индексу (index scan), наполненной малым количеством данных (1000 записей):

```
1 SELECT * FROM test_idx_tbl where int = $1
```

Тестовые таблицы имели следующий формат:

- test_table:
 - int_field : INTEGER PRIMARY KEY

- another_field : BIGINT
- test_tbl1:
 - id1 : INTEGER PRIMARY KEY
 - str1 : TEXT
 - ref : INTEGER
- test_tbl2:
 - id2 : INTEGER PRIMARY KEY
 - str2 : TEXT
- test_idx_tbl:
 - id : INTEGER PRIMARY KEY
 - str : TEXT
 - int : INTEGER

В варианте теста, где производилась выборка по индексу, на данную таблицу навешивался неуникальный индекс по полю int.

Для тестирования использовался сервер БД PostgreSQL 9.2 (из репозитория CentOS 7) с настройками по умолчанию. БД sqlite использовалась так же с настройками по умолчанию.

Тестирование производилось на компьютере под управлением CentOS 7 (x86-64), Intel Core i5 3.1GHz, 16 ГБ ОЗУ.

Сервер PostgreSQL работал на той же машине, где и тестовая утилита.

Результаты тестирования

В таблице ниже приведены результаты тестирования. Тестовая утилита прогонялась несколько раз, в таблице приведены лучшие результаты (меньшее время исполнения). В общем случае никакие сторонние факторы не могут привести к ускорению работы программы, а могут лишь замедлить её. Поэтому можно сделать вывод, что большая производительность достигается в случае, когда сторонние факторы оказывали наименьшее влияние и, таким образом, это наиболее корректные результаты.

SQLite, запросов в секунду	PostgreSQL, запросов в секунду	
Insert (1 запись в транзакции)	9.760	112.765
Insert (10 записей в транзакции)	88.528	1083.776
Insert (1000 записей в транзакции)	1646.131	9647.202
Update (1 запись в транзакции)	9.154	107.689
Update (10 записей в транзакции)	80.543	1082.368
Update (1000 записей в транзакции)	2331.274	7725.587
Delete (1 запись в транзакции)	9.365	103.263
Delete (10 записей в транзакции)	82.283	1069.176
Delete (1000 записей в транзакции)	1165.561	7261.531
Select с left join	17093.433	3333.067
Select (seqscan) по большой таблице	11.811	11.105
Select (index scan) по больш. таблице	29.285	97.590
Select (seqscan) по мал. таблице	10241.704	3582.175
Select (index scan) по мал. таблице	19033.118	4933.399

Выводы

Сервер PostgreSQL выдал значительно более высокую производительность на тестах, в которых выполнялись модифицирующие операции. В SQLite на каждую модифицирующую транзакцию тратится минимум 100 мс времени. Это время SQLite затрачивает на сброс данных на диск (судя по официальной документации, он ожидает физической записи данных для исключения возможности потери данных в случае отключения питания). В PostgreSQL, судя по результатам тестов, реализованы более быстрые алгоритмы, обеспечивающие целостность.

На операциях чтения PostgreSQL сильно отстает от SQLite. Однако, судя по разнице в результатах тестов где производилась выборка данных по индексированной и неиндексированной таблицам, можно сделать вывод о том, что в SQLite более медленная реализация индексов – использование индексов в этой БД даже на большой таблице дает всего трехкратный прирост скорости по сравнению с последовательным перебором.